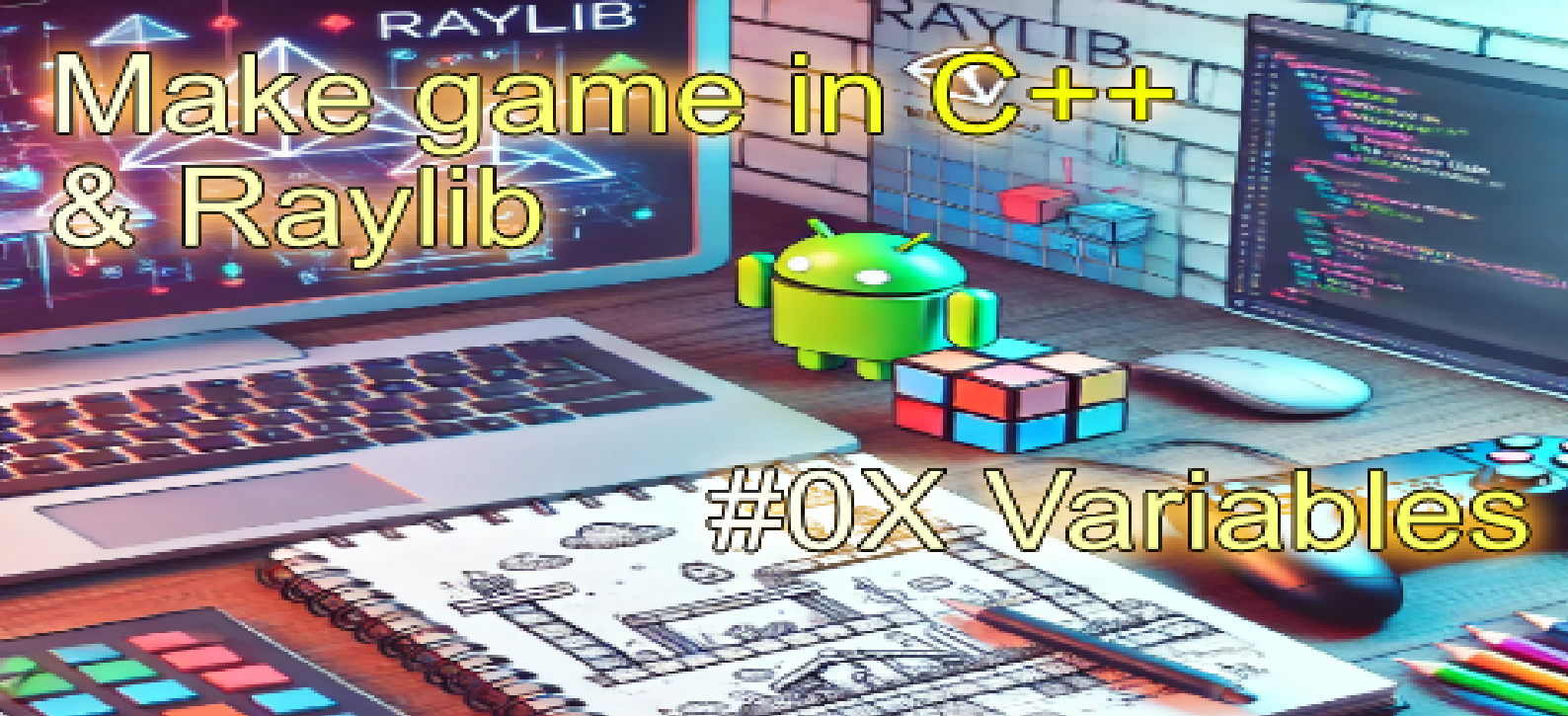




Objectif du module

Nous allons voir rapidement ce qu'est une variable en C++, et surtout les formes les plus importantes (voire indispensables).

Nous verrons ultérieurement les variables les plus importantes de la librairie Raylib (dans l'univers 2d)

Nous apprendrons comment créer une variable, y écrire.

Pour commencer, une variable correspond à un espace mémoire réservé dans l'ordinateur. Cet espace est dédié uniquement à cette variable.

Cependant, vous ne pouvez pas stocker n'importe quoi n'importe où : si l'ordinateur attend des bananes, vous ne pouvez pas lui donner des patates. C'est le principe du typage en C++. (encore qu'il est très con et va faire lui-même ce qu'il croit que vous voulez (yeux qui roulent et écume à la bouche) La programmation n'étant qu'une suite de traitements et de transformations de données, les variables sont donc indispensables.

On peut distinguer deux grandes familles :

- les **simples** (elles stockent une donnée unique)
- les **complexes** (elles regroupent plusieurs données ou suivent un modèle).

Commençons par énumérer les types fondamentaux. Le **format générique** est le suivant :

`typeVariable nomVariable => int age, float tailleEnMètre, bool estVivant`



Théorie

Le typage :

Variables simples :

Int => entier (1,2,3,4,...)

Float => nombre décimale (1,5)

Char => caractère (a,b,c)

Bool => vrai/faux

Variables complexe :

1. array
2. struct

Je vous donne un screen de différentes variables dans l'éditeur de code et un descriptif.

```
#include "raylib.h"
#include <stdio>

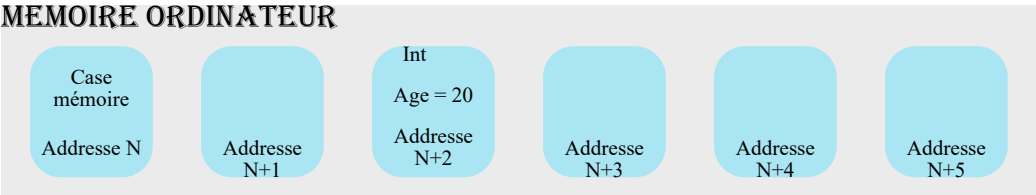
int main(void)
{
    int age = 45;
    float hightMeter = 1.85f;
    bool isActive = true;
    char firstLetter = 'b';
    //array
    int note[5] = {6,8,9,1,4};
    //struct
    struct personnage
    {
        bool isAlive;
        int health;
        float endurance;
    };
    //pointer
    int* intNullPtr = nullptr;
    int intToTarget = 10;
    intNullPtr = &intToTarget;
```

- 👉 Résumé :
- Le **tableau** = plusieurs valeurs **du même type**.
- La **struct** = plusieurs valeurs, **éventuellement de types différents**.

Eh bien dans le code précédent, nous avons instancié les variables et attribué une valeur à ces dernières (ce qui est une bonne pratique même si c'est zéro). Nous allons utiliser ce que l'on appelle des opérateurs pour modifier, comparer les variables. La modification et l'accès aux variables "complexes" se fera

La mémoire :

Il va nous falloir survoler un truc qui fait peur mais qui est essentiel et finalement pas trop compliqué à comprendre mais au début difficile à appliquer. Pour commencer on crée/stocke des données mais où. Bien le mieux c'est de s'imaginer la mémoire comme un parking et chaque place peut contenir une donnée (variable) imaginons la création d'un int age = 20; Voici comment on pourrait schématiser la mémoire.



Et chaque case mémoire dispose d'une adresse (attention c'est là que ça devient fumant). Nous allons voir qu'il est possible de modifier une variable via des opérateurs classiques et facile à prendre en main (voir très intuitif façon calculatrice). Mais le principe de l'adresse est quelque chose utilisé par l'ordinateur. Et en tant que programmeur C++ tout puissant, nous disposons aussi de cet accès (rapide / direct / empreinte mémoire faible). Il s'agit des pointeurs. On va les survoler très rapidement on en aura besoin bientôt. Mais pas de panique vous devez juste savoir ce que c'est et revenir à ce chapitre quand vous y êtes confronté. On les manipulera avec plus de précision et expliquerons leurs intérêts plus tard. Dans la carte sur le **printf()** il y avait un **p%** il sert à afficher l'adresse de la dite case. Et il s'agit d'**hexadécimal** encore une fois pas de panique c'est un code informatique presque matériel. Et si on regarde cette place de parking vue d'en haut cela correspond à des cellules d'un tableau. C'est pourquoi l'adresse pour simplifier est ici notée N et N+1 pour chaque case ou cellule mémoire adjacente. Bien voyons comment initialiser un pointeur.

```
int* intNullPtr = nullptr;
int intToTarget = 10;
intNullPtr = &intToTarget; // add the target adresse to the pointer
```

- int** → un entier, un chiffre rond.
- float** → un nombre à virgule. ⚠ Il faut terminer la valeur par la lettre f.
- bool** → une valeur logique : true ou false, pas d'autre option.
- char** → un caractère simple (lettre, chiffre, symbole), écrit entre ''.
- Tableau** (array) → plusieurs valeurs du même type regroupées.
- struct** → un type modulaire qui peut contenir plusieurs valeurs de types différents, représentant une entité (personnage, livre, voiture, etc).
- Pointeur** → une variable contenant une adresse (ici un int) initialisé à null.
- Adresse** → l'adresse d'une variable existante (ici aussi un int)

```
int* intNullPtr
```

int* est une variable qui peut **stocker l'adresse d'un** int. Il s'agit de l'adresse gps où est garée la voiture (le chiffre).

```
&intToTarget
```

& cible l'**adresse** de la voiture. Donc avec **int*** vous créez une **variable** de type **adresse** pour un **int** et avec **&** vous lui assignez l'**adresse** d'une variable de type int. On dit donc que **int*** => pointe vers **&**. Si vous devez créer un pointeur vide il est **IMPERATIF** de le rendre null sinon vous risquez de tout péter. On peut vraiment citer "De grands pouvoirs impliquent de grandes responsabilités" dans ce cas précis. Et pour essayer de résumer le plus simplement possible je dirais que : Un pointeur est une **variable** qui **stocke l'adresse** d'une **autre variable** de **même type**.

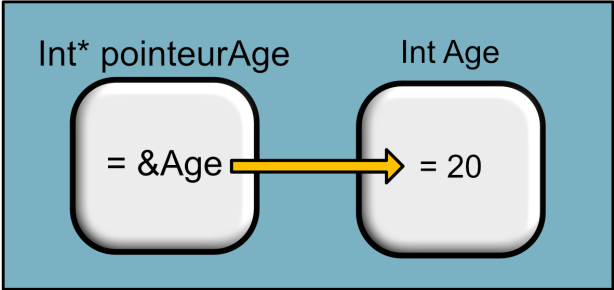
```
int* ptrNull = nullptr;
```

Voici comment initialiser un pointeur vide.

Il y a un concept d'adresse mémoire dynamique mais pour l'heure j'en sais pas plus que vous on apprendra ceci ultérieurement pour infliger des dégâts par exemple. Je vous montre l'image d'une adresse hexadécimale pour information.

```
adresse hexadecimale d'un pointeur : 000000a41b9ffc2c
```

Mais ne bloquez pas sur ces concepts, ils ne sont pas fondamentaux. Mais avec notre méthodologie vous y arriverez sans souci. Il s'agit d'un concept "*easy to do but hard to master*" dans la programmation pro. Ce qui fait des pointeurs et adresses des éléments de maîtrise et il est bien d'essayer de les utiliser pour un code propre et performant. Pour ma part je ne maîtrise pas encore ces concepts mais nous allons les assimiler en parallèle avec notre nouvel Atomic Habit. J'ai essayé de faire un schéma pour représenter un pointeur sur l'image qui suit.



```
*intNullPtr += 10;
```

Le code ci-dessus permet d'utiliser un opérateur (ici l'addition) pour modifier la valeur de la variable pointée. **Attention** * est ici indispensable sinon encore une fois vous risquez de tout péter. C'est un coup à prendre mais ça viendra avec le temps et l'itération des cartes (j'espère 😊).

Les opérateurs :

◆ Opérateurs arithmétiques

Utilisés pour faire des calculs.

Opérateur	Signification	Exemple
+	addition	a + b
-	soustraction	a - b
*	multiplication	a * b
/	division	a / b

◆ Opérateurs d'affectation

Permettent de modifier la valeur d'une variable sans tout réécrire.

Opérateur	Exemple	Correspond à
=	a = 10	Met 10 dans a
+=	a += 5	a = a +5
-=	a -= 3	a = a - 3
*=	a *= 4	a = a * 4
/=	a /= 2	a = a / 2

◆ **Opérateurs de comparaison**

Permettent de comparer deux valeurs (résultat : true ou false).

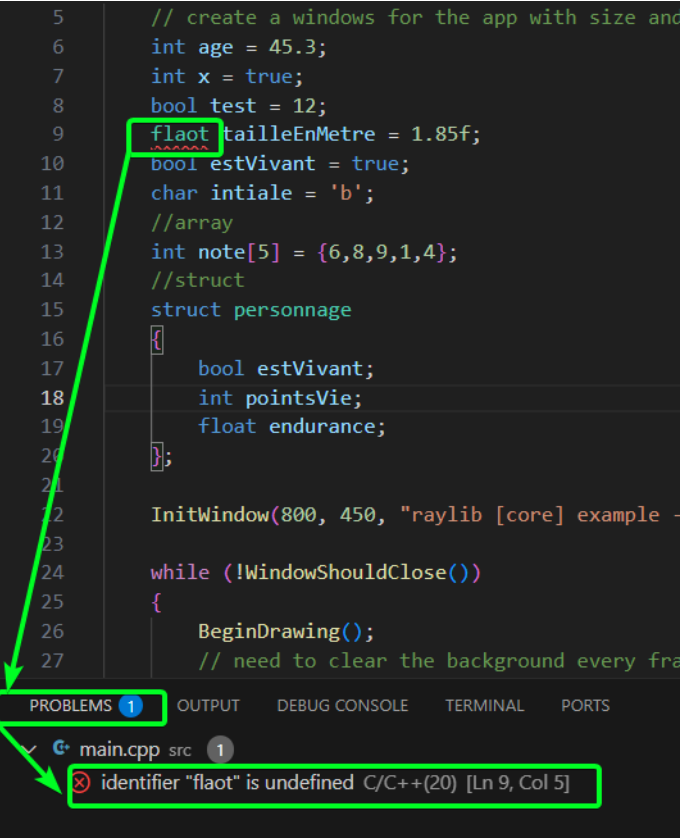
Opérateur	signification	Exemple
==	Égal a	a == b
!=	Différent de	a != b
<=	Inférieur égale à	a <= b
>=	Supérieur égale à	a >= b
<	Strictement inférieur	a < b
>	Strictement supérieur	a > b

◆ **Opérateurs logique**

Permettent de combiner les opérateurs de comparaison

Opérateur	signification	Exemple
&&	et	a > b && a < c (a compris entre b et c)
	ou	a > b c < d

La console :



La [console](#) nous donne des informations importantes. Nous allons la solliciter pour cette flashcard dans la mesure où j'ai généré une erreur de typo relatif à un typage (mouhahaha).

Warnings :

Contrairement à la légende de base le compilateur est très permissif (bonne pratique de le rendre un peu plus rigide pour éviter des bugs cachés). Vous pouvez voir que nous avons des valeurs qui ne correspondent pas du tout au type mais le compilateur fait soit une conversion implicite ([cast](#)) transforme un entier en float par exemple. Vous ne verrez donc pas d'erreur apparaître dans ces cas de figure à vous d'avoir de la rigueur. Mais il s'agira d'une pratique courante/quotidienne de créer et gérer des variables

Bien passons maintenant à l'exercice.

✓ **Résumé**

- Point clé 1 Les variables stockent des données typées en mémoire.
- Point clé 2 Types simples : int, float, char, bool ; complexes : array, struct.
- Point clé 3 Opérateurs pour calculs, affectations et comparaisons.

💡 **À retenir**

- Règle importante Toujours initialiser les variables pour éviter les bugs.
- Piège à éviter Conversions implicites qui masquent des erreurs de typage.
- Bonne pratique Utiliser des noms de variables descriptifs comme tailleEnMètre.

🎯 **Validation des compétences**

- L'élève sait déclarer et typer correctement une variable.
- L'élève sait utiliser des opérateurs arithmétiques et d'affectation.
- L'élève comprend le concept de mémoire et pointeurs basiques.

📖 **Définition :**

[cast](#) : Un [cast](#) (ou transtypage) est l'opération qui permet de convertir une valeur d'un type de donnée en un autre type. (explication et utilisation ultérieurement dans la formation)

[Console](#) : élément de l'éditeur qui essaye de communiquer avec nous tant bien que mal.

[Hexadécimal](#) :

🔧 **Partie pratique**

Il faudra relire toutes les variables et leur attribuer au choix le bon type ou une valeur correspondante au type existant et corriger les typos s'il y a lieu. Création et modification de variables (cf énoncé de la carte)